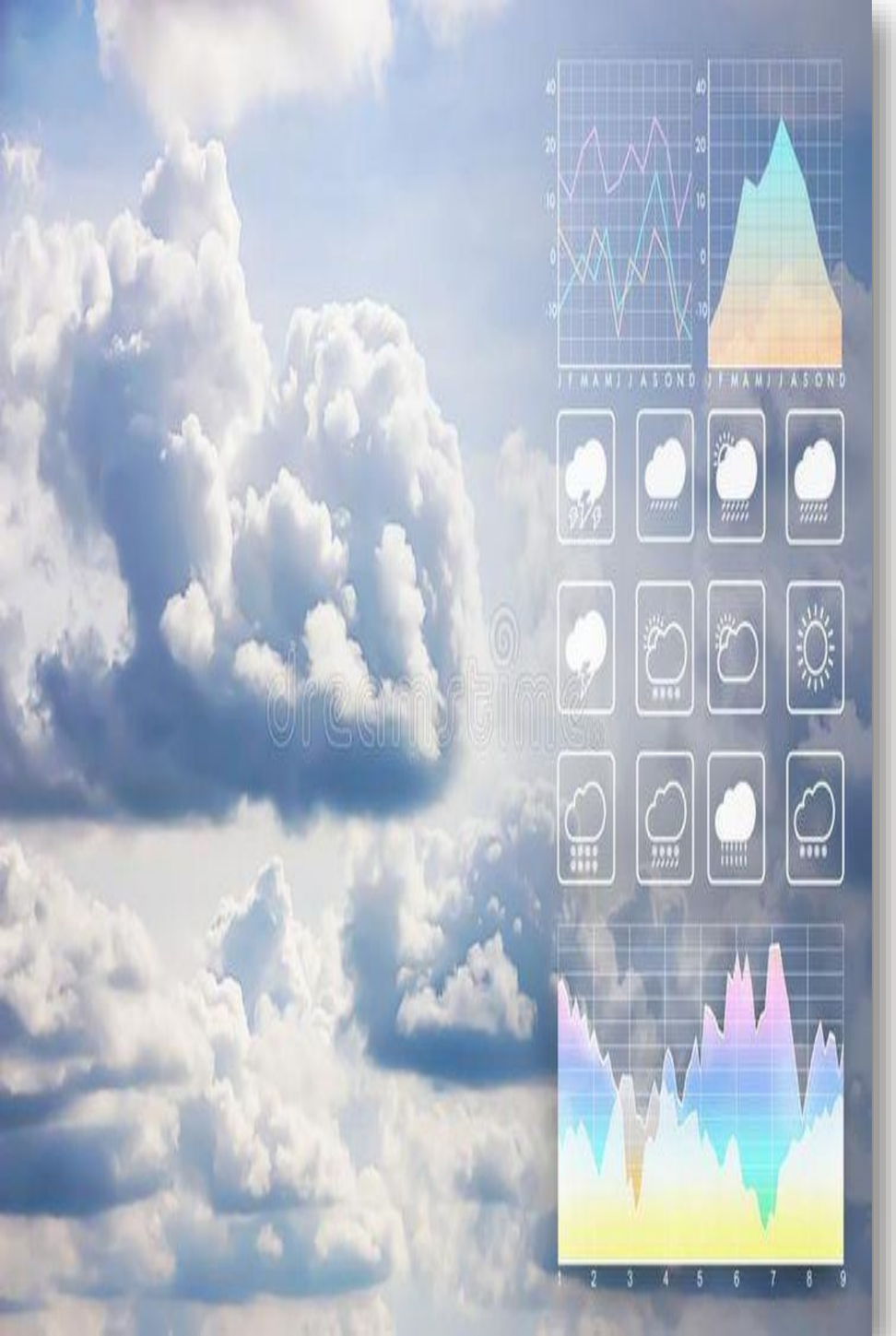


Weather Chatbot: A Python Mini-Project

A simple chatbot that brings live weather updates right to your terminal.

Project Introduction

What Does the Chatbot Do?

The Weather Chatbot is a concise, command-line application built in Python. It offers a quick and easy way for users to check real-time weather conditions for any major city across the globe.

- *User-friendly command-line interface.*
- *Global city coverage via a powerful weather API.*
- *Instant updates on key metrics.*

Temperature

Current heat status.

Humidity

Atmospheric moisture level.

Weather Status

Clear, cloudy, rain, etc.

Core Objectives

This project was primarily a practical exercise, designed to apply foundational Python skills to a real-world scenario—interacting with web services and processing external data.

1

Interactive Development

To successfully develop a simple, interactive chatbot using core Python features and principles of conversational design.

2

API Integration Mastery

To learn and demonstrate proficiency in integrating external RESTful APIs (Application Programming Interfaces) into a Python application.

3

JSON Data Handling

To practise parsing and extracting specific information from data structures returned in JavaScript Object Notation (JSON) format.

Tools and Technology Stack

The chatbot relies on a lean and modern stack, focusing on widely used open-source libraries and development environments.

1

Python 3

The core programming language for the entire application logic.

2

Requests

Essential library for making HTTP calls to the external API.

3

OpenWeatherMap

The source for accurate, real-time weather data.

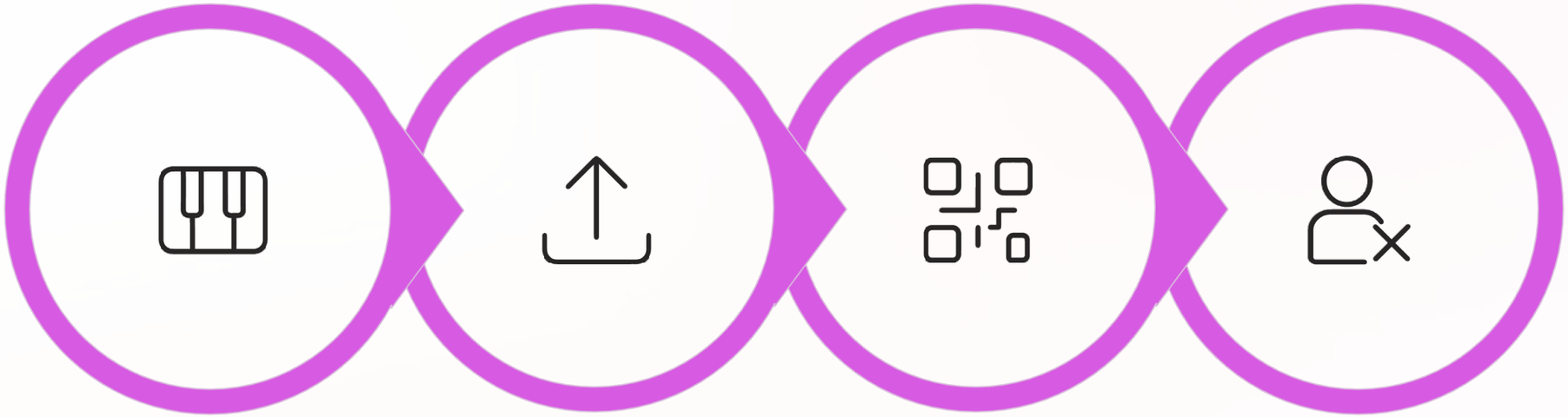
4

VS Code

The primary integrated development environment (IDE) used for coding and debugging.

The Chatbot's Operational Flow

The project follows a linear process, transforming a simple text input into structured, meaningful data presented back to the user.



User enters a city name (e.g., 'London').

Chatbot constructs and sends a request to the OpenWeatherMap API.

API returns the real-time weather data in a raw JSON format.

Chatbot parses the JSON and displays the clean summary.

Core Code Logic: The API Call

Weather Chatbot

main.py

main.py > ...

```python
import requests

API_KEY = Your API here
BASE_URL = "https://api.openweathermap.org/data/2.5/weather"

def get_weather(city):
    params = {
        "q": city,
        "appid": API_KEY,
        "units": "metric"
    }
    response = requests.get(BASE_URL, params=params)
    data = response.json()

    if data.get("cod") != 200:
        print("⚠️ City not found! Please try again.")
    else:
        main = data["main"]
        weather = data["weather"][0]["description"]
        print(f"\n🌆 City: {city}")
        print(f"🌡️ Temperature: {main['temp']}°C")
        print(f"💧 Humidity: {main['humidity']}%")
        print(f"☁️ Weather: {weather.capitalize()}")

print("🌦️ Welcome to Weather Chatbot!")

while True:
    city = input("\nEnter city name (or 'exit' to quit): ")
    if city.lower() == "exit":
        print("👋 Goodbye!")
        break
    get_weather(city)
```

Made with GAMMA

Real-World Sample Output

This example demonstrates the clean, concise information provided by the chatbot, ensuring maximum readability for the user.

The final output is formatted clearly in the terminal, extracting only the most important data points from the complex JSON response.

- *City name confirmation.*
- *Current temperature (in °C).*
- *Humidity percentage.*
- *General weather description.*

User Input

`Enter City Name: Karachi`

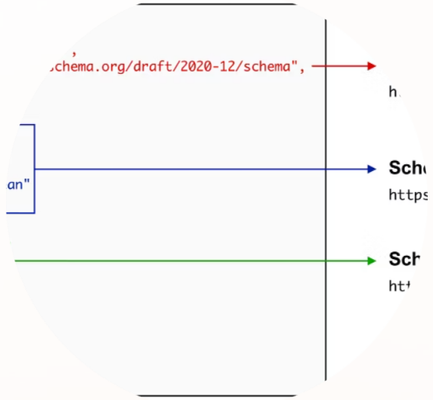
```
PS C:\Users\HP\Desktop\Weather Chatbot> & C:/Users/HP/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/HP/Desk
top/Weather Chatbot/main.py"
🌦Welcome to Weather Chatbot!

Enter city name (or 'exit' to quit): karachi

🌆 City: karachi
🌡Temperature: 26.9°C
💧 Humidity: 39%
☁Weather: Haze
```

Challenges and Key Learnings

Building the chatbot provided crucial hands-on experience in dealing with common development hurdles in data-driven applications.

API Structures

Gaining a deeper understanding of how to correctly format API request URLs, including parameters and API keys, and handling response codes.

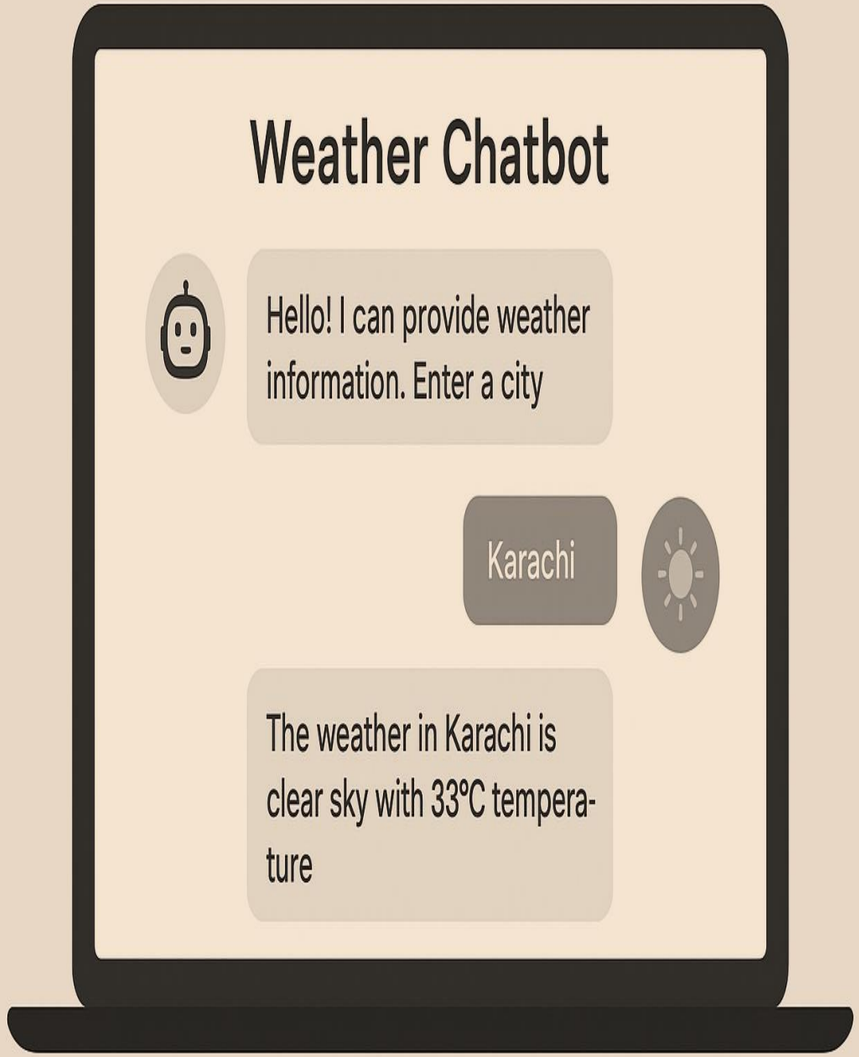


Parsing JSON

Mastering the technique of navigating nested dictionary and list structures within JSON data to precisely extract required values.

Robust Error Handling

Implementing `try-except` *blocks to gracefully manage scenarios like network errors or invalid user input (e.g., non-existent city names).*



Next Steps: Future Improvements

The current project serves as a strong foundation, but there are several planned enhancements to boost its functionality and user experience.

Desktop GUI

Transitioning from the terminal to a Graphical User Interface using libraries like Tkinter or Streamlit for a more accessible visual display.

Extended Forecast

Integrating multi-day forecast data (e.g., a 5-day outlook) to provide predictive information alongside the current weather.

Voice Interaction

Adding support for voice input and output, transforming the chatbot into a genuinely interactive voice assistant for weather queries.

Conclusion: Key Takeaways

The Weather Chatbot project proved to be an invaluable exercise in bridging theoretical knowledge with practical application.

Python Versatility

Confirmed Python's role as an incredibly versatile tool for quick prototyping and utility scripting.

Real-World Data

Successfully handled, processed, and presented real-world, dynamic data from an external source.

API Ecosystem

Developed confidence in navigating the API ecosystem, which is essential for modern software development.

> *The greatest learning was realizing how a few lines of Python code, combined with a public API, can create a genuinely useful application.*